

Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

Sistem RAG pentru extragerea relațiilor între entități

Proiect de diplomă

prezentat ca cerință parțială pentru obținerea
titlului de *Inginer*
în domeniul *Electronică și Telecomunicații*
programul de studii *Electronică Aplicată*

Conducător științific
Ș.l. Dr. Ing. Liviu-Daniel ȘTEFAN
Drd. Fiz. Alexandru IONIȚĂ

Absolvent
Andreas BAȘCHIR

Anul 2026

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul *Sistem RAG pentru extragerea relațiilor între entități*, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității Naționale de Știință și Tehnologie POLITEHNICA București ca cerință parțială pentru obținerea titlului de *Inginer* în domeniul *Electronică și Telecomunicații*, programul de studii *Electronică Aplicată* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate.

Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare.

Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Iunie 2026.

Absolvent: Andreas BAȘCHIR

.....

Cuprins

Lista figurilor	7
Lista tabelelor	9
Lista acronimelor	11
Introducere	12
1. Studiu de literatură	14
1.1. Prezentarea domeniului lucrării de diplomă	14
1.1.1. Introducere	14
1.1.2. Problema: Cunoștințe statice care duc la halucinații	14
1.1.3. Soluția: Retrieval-Augmented Generation	14
1.1.4. Formularea problemei: Extragerea relațiilor între entități	15
1.1.5. Relevanță și aplicații	15
1.2. Taxonomie	16
1.2.1. Motivația definirii unei taxonomii	16
1.2.2. Criterii de clasificare	16
1.2.3. Categoriile principale	17
1.3. Arhitectura generală a unui sistem de tip RAG	17
1.3.1. Componentele sistemului	17
1.3.2. Fluxul de date	18
1.4. Metode și modele RAG	18
1.4.1. Naive RAG	18
1.4.2. Advanced RAG	19
1.4.3. Hypothetical Document Embeddings (HyDE)	19
1.4.4. REALM, Fusion-in-Decoder și RETRO	19
1.4.5. RAPTOR	19
1.4.6. Corrective RAG (CRAG)	19
1.4.7. Self-RAG	20
1.4.8. Adaptive-RAG și FLARE	20
1.4.9. GraphRAG și HippoRAG	20
1.4.10. RAG pentru contexte lungi și aplicații specializate	20
1.5. Metrice de evaluare	21
1.5.1. Metrice de recuperare	21
1.5.2. Metrice de generare	21
1.5.3. Metrice specifice sistemelor RAG	21
1.5.4. Dificultăți în evaluarea extragerii relațiilor	22
1.6. Baze de date	22
1.6.1. Seturi de date pentru sisteme RAG	22
1.6.2. Seturi de date pentru extragerea relațiilor	23

1.7. Limitări și provocări	23
1.8. Concluzii	24
2. Rezultate experimentale	25
2.1. Analiză cantitativă	26
2.1.1. Configurarea experimentului	26
2.1.2. Rezultate	27
2.1.3. Semnificație statistică	27
2.2. Analiză calitativă	28
2.2.1. Compromisul dintre calitate și latență	28
2.2.2. Unde ajută recuperarea hibridă	29
2.2.3. Decalajul dintre recuperare și generare	29
2.2.4. Constrângerea de groundare și refuzurile	30
2.2.5. Observații calitative din interfața Streamlit	30
Concluzii	32
Anexa 1.	37
Anexa 2.	38

Lista figurilor

1.1.	Evoluția modelelor lingvistice mari (LLMs) din 2018 până în 2024	15
1.2.	Arhitectura generală a unui sistem RAG: indexare offline (sus) și inferență online (jos) [2, 5].	18
2.1.	Metrice de recuperare: Context Recall@5 și MRR pentru Naive RAG și Advanced RAG. Barele de eroare reprezintă deviația standard calculată pe cele 4 seed-uri.	27
2.2.	Calitatea răspunsurilor: Answer F1 și Exact Match pentru Naive RAG și Advanced RAG.	28
2.3.	Latența medie de procesare a interogării în milisecunde. Latența Advanced RAG include apelul LLM pentru expansiunea interogării, recuperarea hibridă paralelă și re-scorarea cu cross-encoder.	29
2.4.	Vedere de ansamblu comparativă a metricilor de calitate (Context Recall@5, MRR, Answer F1, Exact Match). Latența nu este inclusă deoarece se află pe o scară diferită (milisecunde față de 0–1).	30

Lista tabelelor

1.1. Comparație a seturilor de date pentru evaluarea RAG și extragerea relațiilor [25–28, 30–35].	23
2.1. Rezultate comparative Naive RAG vs. Advanced RAG pe SQuAD ($N = 600$ per sistem, medie $\pm$ deviație standard pe 4 seed-uri)	27
2.2. Teste de semnificație statistică (Advanced RAG vs. Naive RAG, $N = 600$ perechi)	28

Introducere

Scopul lucrării de diplomă

Sistemele RAG (Retrieval-Augmented-Generation) reprezintă o arhitectură de sistem de inteligență artificială care combină recuperarea informațiilor relevante din surse externe cu generarea de text de către modele de limbaj mari (LLMs - Large Language Models)

RAG a fost propus ca soluție pentru natura statică a informațiilor reținute de către modelele de limbaj mari, consecință a antrenării pe durate limitate, cu pauze între versiuni.

Gradul de noutate

Sistemele RAG au apărut oficial în 2020, prin lucrarea „Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks” publicată de Lewis et al. de la Facebook AI Research (astăzi Meta AI), în contextul exploziei modelelor transformer precum BERT și GPT, care generau răspunsuri creative, dar adesea inexacte sau „halucinate” din cauza limitărilor cunoștințelor lor statice.

De la începutul în 2020 cu RAG „naive”, sistemul a evoluat rapid: în 2023-2024 au apărut variante precum Advanced RAG și Modular RAG; în 2024, Microsoft a introdus GraphRAG pentru relații complexe via knowledge graphs, iar în 2025 s-au impus hibridări (vectori denși și rari, SQL-RAG pentru operații matematice) și Multi-RAG pentru scalabilitate în întreprinderile mari. Până în 2026, RAG este omniprezent în aplicații de business, chatbots și agenți AI, fiind o tehnologie utilizată în producție, trecută de ani de stadiul de prototip.

Obiectivele proiectului

Obiectivul proiectului este de a introduce noțiuni despre contextul utilizării sistemelor RAG, cum se pot implementa, care sunt constrângerile dar și punctele forte ale acestora. Se vor prezenta și posibile direcții de viitor ale domeniului sistemelor RAG.

Contribuții și rezultate

Se va prezenta o comparație utilizând metrici din mediul academic dintre un sistem RAG de tip ”naive” și un sistem de tip ”advanced”, cu o interfață grafică de testare a LLM-urilor.

Conținutul lucrării

Lucrarea este structurată în două capitole principale:

Capitolul 1 - Studiu de literatură parcurge domeniul RAG din perspective teoretice și practice. Acesta cuprinde: (1) introducerea și contextul LLM-urilor în extragerea informației; (2) taxonomia sistemelor RAG clasificate după tipul recuperării, integrării și surselor de informație; (3) arhitectura generală a unui sistem RAG cu descrierea componentelor principale; (4) prezentarea metodelor și modelelor reprezentative din literatură (Self-RAG,

CRAG, HyDE, GraphRAG); (5) definirea și analiza metricilor de evaluare pentru fiecare etapă a pipeline-ului de RAG (recuperare, generare, augmentare); (6) prezentarea seturilor de date utilizate în evaluarea RAG (T-REx, CRAG, WebNLG, HotpotQA); și (7) discutarea limitărilor și provocărilor actuale ale sistemelor RAG.

Capitolul 2 - Rezultate experimentale prezintă implementarea și comparația empirică a două tipuri de sisteme RAG: (1) Naive RAG - versiunea de bază cu recuperare pe bază de vectori denși și generare de către limbajul lingvistic mare; și (2) Advanced RAG - cu recuperare de tip hibrid, reordonare și rescriere a interogărilor LLM-ului. Pentru experimente a fost utilizat setul de date T-REx (relații entitate-entitate din Wikipedia aliniate cu Wikidata). Capitolul cuprinde atât analiza cantitativă (metrici RAGAS, Precision@K, Recall@K, MRR, F1) cât și analiza calitativă ale sistemelor RAG. Sistemele ilustrate anterior pot fi testate interactiv într-o interfață implementată în Streamlit.

Capitolul 1

Studiu de literatură

1.1 Prezentarea domeniului lucrării de diplomă

1.1.1 Introducere

Revoluția modelelor bazate pe arhitectura transformer, apărută în 2017 [1] și a limbajelor lingvistice mari (Large Language Models - LLMs) a dus la o schimbare paradigmatică a abordării problemelor de procesare de limbaj natural. Modele precum BERT și GPT-3 au demonstrat capacități remarcabile în înțelegerea și generarea de text coerent, ceea ce a dus la o adoptare rapidă a LLM-urilor global, cercetarea despre LLM-uri devenind un punct central de interes. În Figura 1.1 se pot observa modelele SOTA de-a lungul anilor 2018-2024. Totuși, LLM-urile de sine stătătoare au o limitare fundamentală, cunoștințele lor sunt fixe, limitate la momentul antrenării, neputând fi actualizate fără a fi reantrenate [2].

1.1.2 Problema: Cunoștințe statice care duc la halucinații

Fenomenul de „halucinație” reprezintă un răspuns aparent bine formulat, dar factual incorect, al unui LLM. Modelele generează text pe baza distribuției de probabilități a datelor cu care au fost antrenate, nu pe baza unor cunoștințe verificate. Pentru utilizatorul obișnuit, halucinațiile nu reprezintă un impediment major, însă există aplicații în care acestea sunt inacceptabile (medical, legal, financiar).

În plus, informația încorporată în parametrii modelului este de natură „statică”, reflectă cunoștințele cunoscute la momentul antrenării. După antrenare, singura metodă de actualizare a informației este procesul costisitor de fine-tuning.

1.1.3 Soluția: Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) este un tip de arhitectură de sistem de inteligență artificială care combină recuperarea informației relevante din surse externe cu capacitatea de generare a LLM-urilor. În contextul unui sistem RAG, documentele relevante sunt recuperate dintr-o sursă externă de date, încorporate în contextul LLM-ului, iar în final, se generează un răspuns. Aceasta adresează ambele probleme simultan:

1. Furnizează informația relevantă pentru un context factual corect, reducând astfel halucinațiile;
2. Permite modificarea cunoștințelor și extensia contextului fără fine-tuning sau reantrenare.

Termenul „RAG” a fost introdus în 2020 de către Patrick Lewis și colaboratorii săi de la Facebook AI Research (astăzi Meta AI) în lucrarea „*Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*” [3].

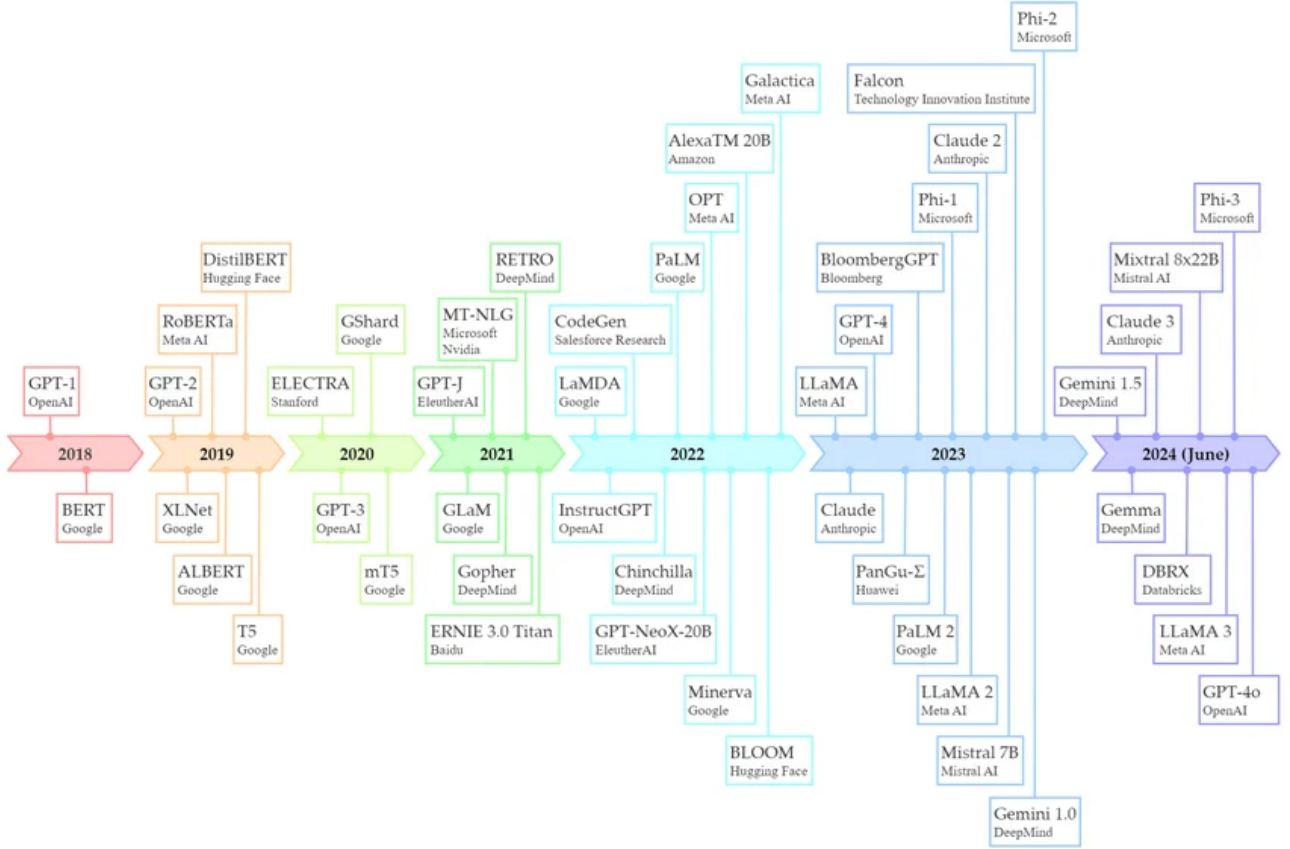


Figura 1.1: Evoluția modelelor lingvistice mari (LLMs) din 2018 până în 2024

1.1.4 Formularea problemei: Extragerea relațiilor între entități

Extragerea relațiilor între entități este o sarcină de bază în procesarea limbajului natural. Dat fiind un text, scopul este de a identifica entitățile menționate și relațiile dintre ele. De exemplu, din textul „Albert Einstein s-a născut în Ulm în 1879”, sistemul trebuie să extragă relațiile (Albert Einstein, născut în, Ulm) și (Albert Einstein, născut în an, 1879).

Formal, fie q o întrebare și $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$ o mulțime de documente. Un sistem RAG acționează în două etape. Prima etapă: un model de recuperare $p_\eta(\cdot | q)$ alege o submulțime $\mathcal{Z} \subseteq \mathcal{D}$ de documente-candidat. A doua etapă: un model lingvistic mare generativ p_θ generează răspunsul y condiționat de întrebare dar și de documentele recuperate:

$$p(y | q) = \sum_{z \in \mathcal{Z}} p_\eta(z | q) p_\theta(y | q, z). \quad (1.1)$$

RAG aduce o alternativă rețelelor neuronale de clasificare create pentru sarcini specifice. Sistemul recuperează pasaje relevante pentru fiecare pereche de entități, iar LLM-ul extrage relația pentru contextul recuperat, astfel generalizând pentru relații noi, fără reantrenare [2].

1.1.5 Relevanță și aplicații

Sistemele RAG sunt relevante într-o gamă largă de domenii unde accesul dinamic la informații externe este esențial pentru generarea de răspunsuri de calitate [2].

- **Construirea grafurilor de cunoștințe:** Wikipedia, DBpedia, Freebase și alte baze de date semantice structurate sunt construite parțial prin extragerea automată a relațiilor

- **Asistenți inteligenți:** Chatbots și agenți de inteligență artificială care necesită cunoștințe actualizate constant
- **Domenii specializate:** Bioinformatică (extragerea relațiilor proteină-proteină), drept (identificarea precedentilor), medicină (relații medicament-boală)
- **Indexare și căutare:** Motoarele de căutare moderne folosesc RAG pentru a livra răspunsuri directe

1.2 Taxonomie

1.2.1 Motivația definirii unei taxonomii

Domeniul RAG s-a dezvoltat rapid, producând numeroase variante și îmbunătățiri ale arhitecturii originale. Identificarea similarităților și diferențelor între abordări este dificilă fără o clasificare sistematică. Lucrările Gao et al. (2024) [2] și Peng et al. (2025) [4] au fost folosite ca referință pentru această secțiune, care introduce o taxonomie bazată pe aspectele tehnologice cheie. Trebuie reținut că taxonomia este limitată la specificul acestei lucrări (RAG pentru extragerea relațiilor) și nu acoperă întreg domeniul.

1.2.2 Criterii de clasificare

1. Tipul mecanismului de recuperare

- **Sparse retrieval:** Metode tradiționale de recuperare de IR (Information Retrieval) precum BM25, care se bazează pe potrivirea cuvintelor cheie. Rapid dar sensibil la variații lexicale și sinonime.
- **Dense retrieval:** Folosește reprezentări numerice a textului din documente (și interogarea către LLM) în spații vectoriale continue. Poate captura nuanțele semantice, dar costul computațional este mai ridicat.
- **Hybrid retrieval:** Combină sparse și dense prin fuzion (ex. Reciprocal Rank Fusion), exploataând avantajele ambelor abordări.

2. Modul de integrare între recuperare și generare

- **Modular:** Componentele sunt decuplate. Recuperatorul și generatorul sunt independente și pot fi optimizate separat, oferind flexibilitate maximă.
- **End-to-end:** Recuperarea și generarea sunt cuplate, și sunt optimizate laolaltă, de regulă prin antrenare (gradient descent).

3. Tipul sursei de cunoștințe

- **Nestructurată:** Documente text din diverse surse precum site-uri web (ex. articole Wikipedia), PDF-uri, documente Word. Pentru a putea fi procesate, necesită indexare și analiză semantică.
- **Structurată:** Grafuri de cunoștințe cu entități și relații explicit modelate (DBpedia, Wikidata, Freebase), permitând navigare relațională.
- **Hibridă:** Combinație ale amândurora, exploataând avantaje complementare.

4. Etapa augmentării

- **Pre-retrieval:** Optimizarea pre-recuperare prin extinderea și/sau reformularea întrebării și descompunere semantică.
- **At-retrieval:** Optimizarea la momentul recuperării, prin selectarea parametrilor k , strategii adaptive de recuperare.
- **Post-retrieval:** Optimizare după recuperare, prin reranking, compactarea contextului și filtrare.

1.2.3 Categoriile principale

Naive RAG

Paradigma originală din lucrarea Lewis et al. 2020 [3]: indexare, urmată de recuperare de tip dense și generare. Recuperatorul și generatorul sunt elemente separate, și nu se folosesc optimizări. Este arhitectura de bază, folosită ca referință în majoritatea studiilor [3], incluzând această teză.

Advanced RAG

Adaugă optimizări pre- și post-recuperare, precum: rescriere a întrebărilor, recuperare hibridă, și reranking cu model special antrenat. Dovedește îmbunătățiri semnificative, între 5 și 15% pe benchmark-uri, cu costuri computaționale moderate. Lucrarea de față va implementa acest tip de arhitectură.

Modular RAG

Diferă de metodele Naive și Advanced în structură, fiecare element din arhitectura Modular RAG este un modul separat (ex. recuperare, memorare, rutare, predicție, căutare). Este cea mai adaptabilă/flexibilă arhitectură, însă introduce și cel mai înalt nivel de complexitate.

Graph RAG

Specifică pentru surse structurate de date: indexare pe grafuri de cunoștințe, recuperare utilizând relații semantice, generare pe bază de subgrafuri. Ideal pentru extragerea relațiilor din grafuri de cunoștințe dar necesită structurare prealabilă a datelor. Descriș în detaliu de Peng et al. (2025) [4].

1.3 Arhitectura generală a unui sistem de tip RAG

Un sistem RAG funcționează în două faze: o fază offline de indexare și o fază online de inferență. Componentele principale sunt întrebarea utilizatorului, baza de cunoștințe, mecanismul de recuperare și modelul generativ [2].

1.3.1 Componentele sistemului

Întrebarea reprezintă nevoia de informație a utilizatorului. În formele de bază, textul întrebării este transmis direct mecanismului de recuperare. Sistemele avansate aplică mai întâi transformări ale întrebării: reformulare semantică, extindere cu termeni sinonimi sau descompunere în sub-întrebări, pentru a crește acoperirea și a reduce sensibilitatea față de formulări ambigue [2].

Baza de cunoștințe este sursa externă din care se extrage informația. Documentele trec printr-o etapă de indexare offline: sunt împărțite în fragmente (eng. *chunks*), codificate cu un model de reprezentare vectorială bazat pe arhitectura transformer [1] și stocate într-o bază de date vectorială [5]. Strategia de fragmentare influențează direct calitatea recuperării:

fragmentele prea scurte pierd contextul necesar pentru identificarea relațiilor între entități, iar cele prea lungi diluează semnalul relevant.

Mecanismul de recuperare punctează fragmentele din baza vectorială în raport cu interogarea și returnează primele k rezultate. Sistemele dense calculează similaritatea cosinus între vectorul interogării și vectorii fragmentelor [5]. Sistemele sparse aplică potrivire ponderată de termeni, BM25 reprezentând standardul din domeniu [6]. Sistemele hibride combină scorurile dense și sparse prin Reciprocal Rank Fusion (RRF) [7]. Opțional, un model de reranking de tip cross-encoder poate rafina ordinea inițială a fragmentelor recuperate.

Modelul generativ primește contextul augmentat, format din interogarea originală și fragmentele recuperate, și produce răspunsul final [3]. Arhitecturile orientate spre fidelitate constrâng explicit modelul să fundamenteze răspunsurile pe dovezile recuperate, reducând astfel rata de halucinații [8].

1.3.2 Fluxul de date

Figura 1.2 ilustrează cele două trasee ale fluxului de date. Offline, documentele sunt procesate o singură dată: fragmentate, codificate și stocate în baza vectorială. Online, fiecare interogare parcurge procesarea interogării, recuperarea fragmentelor, augmentarea contextului și generarea răspunsului.

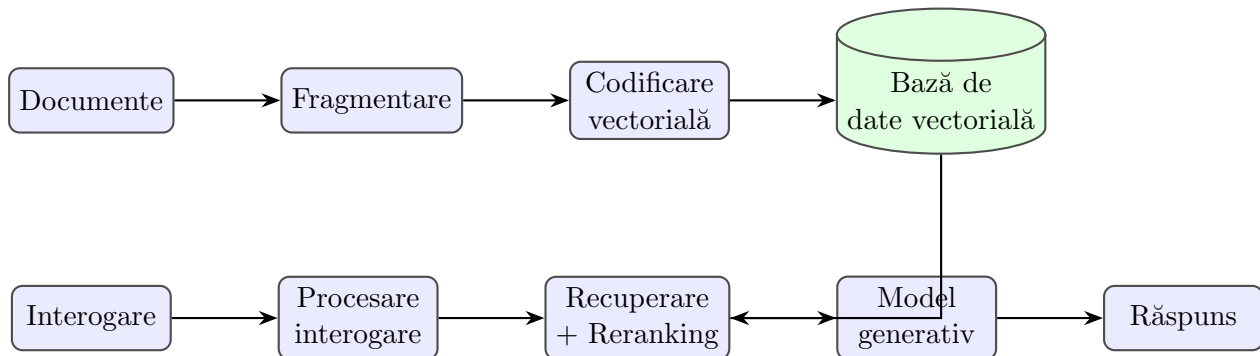


Figura 1.2: Arhitectura generală a unui sistem RAG: indexare offline (sus) și inferență online (jos) [2, 5].

1.4 Metode și modele RAG

1.4.1 Naive RAG

Formularea originală a RAG, propusă de Lewis et al. (2020) [3], combină un recuperator bazat pe Dense Passage Retrieval (DPR) cu un model generativ de tip sequence-to-sequence, optimizate parțial în comun. Fluxul de lucru este liniar: indexare, recuperare densă, generare. Această variantă servește ca referință în majoritatea studiilor din literatura de specialitate [2].

Limitările Naive RAG sunt bine documentate: sistemul nu gestionează variațiile lexicale (sinonime, parafrazări), nu evaluează calitatea documentelor recuperate și nu filtrează fragmentele irelevante înainte de generare [2]. Cu toate acestea, simplitatea sa face din Naive RAG un punct de plecare util pentru orice evaluare comparativă, inclusiv în cadrul acestei teze.

1.4.2 Advanced RAG

Advanced RAG adaugă optimizări pre-recuperare și post-recuperare față de varianta de bază [2]. Pre-recuperare: fragmentare semantică, îmbogățirea fragmentelor cu metadate, generare de interogări multiple. Post-recuperare: reranking cu model cross-encoder, compactarea contextului pentru a reduce zgomotul din prompt. Scorurile dense și sparse sunt combinate prin RRF [7]. Studiile raportează îmbunătățiri de 5–15% față de Naive RAG pe benchmark-uri standard.

Pentru extragerea relațiilor, optimizările post-recuperare sunt relevante în mod direct: identificarea corectă a unei relații necesită fragmente care conțin simultan ambele mențiuni de entitate și indiciile lexicali ai relației. Reranking-ul crește șansa ca aceste fragmente să ajungă în contextul modelului generativ. Aceasta este arhitectura pe care lucrarea de față o implementează și evaluează în partea experimentală.

1.4.3 Hypothetical Document Embeddings (HyDE)

HyDE [9] folosește un model de limbaj pentru a genera un pseudo-document care ar conține răspunsul la interogare. Acest pseudo-document este codificat și se caută documente reale similare prin recuperare densă. Encoderile contrastive eliminată erorile factuale din pseudo-document, păstrând tiparele de relevanță semantică [9].

Avantajul principal al HyDE este că îmbunătățește recuperarea fără a necesita date de antrenare etichetate pentru interogări. Dezavantajul este dependența de calitatea generării inițiale: un pseudo-document slab generat poate conduce recuperarea în direcția greșită.

1.4.4 REALM, Fusion-in-Decoder și RETRO

REALM [10] integrează recuperarea chiar în etapa de pre-antrenare, tratând recuperatorul ca o variabilă latentă optimizată împreună cu modelul de limbaj. Abordarea arată că un recuperator co-antrenat cu generatorul înțelege mai bine ce tipuri de documente sunt utile pentru diferite sarcini.

Fusion-in-Decoder (FiD) [11] codifică fiecare pasaj recuperat independent prin encoder, dar realizează fuziunea la nivelul decoderului. Astfel, modelul poate agrega dovezi din surse multiple fără a sufoca encoderile cu un context prea lung. RETRO [12] extinde această idee la scara corpusurilor de zeci de trilioane de tokenuri, fără a crește proporțional numărul de parametri ai modelului, obținând eficiență parametrică pe seama unei infrastructuri mai complexe.

1.4.5 RAPTOR

RAPTOR [13] construiește un arbore de rezumate de jos în sus: fragmentele de text sunt grupate în clustere, rezumate, iar procesul se repetă recursiv, generând niveluri de abstractizare din ce în ce mai înalte. La inferență, recuperarea operează pe acest arbore și poate agrega informații de la mai multe niveluri de abstractizare simultan.

Metoda obține rezultate bune pe sarcini care necesită raționament distribuit pe mai multe documente [13]. Avantajul față de recuperarea standard este că furnizează modelului generativ atât detalii de granularitate fină, cât și context global. Dezavantajul constă în costul de construire și actualizare a arborelui.

1.4.6 Corrective RAG (CRAG)

CRAG [14] adaugă un evaluator al calității documentelor recuperate. Când relevanța fragmentelor este insuficientă, sistemul declanșează acțiuni corective: recuperare suplimentară

sau căutare pe web, urmată de rafinarea cunoștințelor prin descompunere și recompunere a informației. Dacă documentele recuperate sunt de bună calitate, fluxul rămâne cel standard.

Mecanismul de auto-corecție face sistemul mai robust față de recuperările greșite și reduce riscul extragerii unei relații dintr-un context irelevant. CRAG este una dintre primele abordări care tratează explicit eșecul recuperării ca un caz de gestionat, nu ca o excepție rară.

1.4.7 Self-RAG

Self-RAG [8] antrenează un singur model să decidă adaptiv când să recupereze, când să genereze și când să critice propriile răspunsuri, folosind tokenuri speciale de reflecție introduse în vocabularul modelului. Modelul evaluează relevanța fragmentului recuperat și gradul de fundamentare factuală a răspunsului generat înainte de a-l returna utilizatorului.

Față de CRAG, care utilizează un evaluator extern, Self-RAG internalizează întregul proces de evaluare și corecție. Fidelitatea și controlabilitatea cresc [8]. Pentru extragerea relațiilor, posibilitatea de a verifica dacă tipul de relație prezis este susținut de dovezile recuperate este utilă în mod direct, deoarece reduce numărul de relații extrase din fragmente slab relevante.

1.4.8 Adaptive-RAG și FLARE

Adaptive-RAG [15] alege dinamic strategia de recuperare potrivită pentru fiecare interogare, fără recuperare, cu recuperare unică sau cu recuperare iterativă, pe baza complexității estimate cu un clasificator mic antrenat separat. Interogările simple nu necesită recuperare; cele complexe, care implică mai mulți pași de raționament, beneficiază de recuperare iterativă.

FLARE [16] abordează problema diferit, prin recuperare activă: modelul generează propoziția următoare și, atunci când detectează tokenuri cu probabilitate scăzută (indiciu de incertitudine), folosește predicția respectivă ca interogare și recuperează documente suplimentare înainte de a regenera propoziția. Recuperarea iterativă din Adaptive-RAG ajută la identificarea relațiilor distribuite pe mai multe documente, iar FLARE reduce halucinațiile în extragerea pe texte lungi.

1.4.9 GraphRAG și HippoRAG

GraphRAG [17] construiește un graf de cunoștințe din corpus prin extragerea entităților și relațiilor, structurând informația în comunități ierarhice rezumate. Recuperarea operează pe acest graf prin traversarea relațiilor semantice, permițând raționamentul global, nu doar căutarea locală de fragmente similare.

HippoRAG [18] integrează un model de limbaj, un graf de cunoștințe și Personalised PageRank: corpusul este transformat într-un graf fără schemă predefinită, iar conceptele din interogare servesc ca noduri de pornire pentru propagarea pe graf. Ambele abordări se înscriu în direcția de cercetare Graph RAG descrisă de Peng et al. (2025) [4]. Avantajul comun este descoperirea relațiilor implicite distribuite în mai multe pasaje, ceea ce le face relevante în mod direct pentru sarcina de extragere a relațiilor. Limitarea comună este costul de construire și actualizare a grafului.

1.4.10 RAG pentru contexte lungi și aplicații specializate

Creșterea numărului de pasaje recuperate nu garantează îmbunătățirea performanței. Pasajele irelevante din contexte lungi degradează calitatea generării, un fenomen documentat de Jin et al. (2024) [19]. Problema selectivității devine la fel de importantă ca problema acoperirii.

Alte extensii documentate în literatura recentă includ RAG multimodal, care integrează documente text, imagini, audio și video [20], MMed-RAG pentru modele vizuale din domeniul medical [21], RAP pentru personalizarea răspunsurilor în funcție de preferințele utilizatorului [22] și abordări de recuperare din colecții de mii de documente [23].

1.5 Metrici de evaluare

Evaluarea unui sistem RAG este mai complexă decât evaluarea unui model generativ izolat, deoarece performanța globală depinde atât de componenta de recuperare, cât și de cea de generare [2]. O recuperare slabă limitează calitatea răspunsului indiferent de capacitățile modelului generativ.

1.5.1 Metrici de recuperare

Precizia la rang k (Precision@ k) este proporția documentelor relevante printre primele k rezultate recuperate:

$$\text{Precision@}k = \frac{|\{\text{relevante}\} \cap \{\text{top-}k\}|}{k}. \quad (1.2)$$

Recuperarea la rang k (Recall@ k) reprezintă proporția tuturor documentelor relevante care au fost efectiv recuperate:

$$\text{Recall@}k = \frac{|\{\text{relevante}\} \cap \{\text{top-}k\}|}{|\{\text{relevante}\}|}. \quad (1.3)$$

Mean Reciprocal Rank (MRR) măsoară rangul primului document relevant, mediat peste $|Q|$ interogări:

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rang}_i}. \quad (1.4)$$

Aceste metrici sunt calculate față de un set de documente relevante adnotate manual. În practică, Recall@ k este adesea mai informativă decât Precision@ k pentru sistemele RAG, deoarece un document relevant ratat nu poate fi recuperat ulterior de modelul generativ [5].

1.5.2 Metrici de generare

Exact Match (EM) este o metrică binară care indică potrivirea completă, token cu token, față de răspunsul de referință. Este strictă și potrivită pentru răspunsuri scurte sau factuale.

Scorul F_1 la nivel de token echilibrează precizia P și recuperarea R față de răspunsul de referință:

$$F_1 = \frac{2 \cdot P \cdot R}{P + R}. \quad (1.5)$$

Pentru răspunsuri mai lungi, unde potrivirea exactă este irealistă, se utilizează suprapunerea de n -grame calculată cu metricile ROUGE și BLEU [2]. ROUGE-L măsoară cel mai lung subșir comun, captând ordinea parțială a tokenurilor.

1.5.3 Metrici specifice sistemelor RAG

Metricile clasice de generare nu surprind fenomenul halucinației în contextul RAG, deoarece nu verifică dacă răspunsul este fundamentat pe contextul recuperat. Cadrul RAGAS [24] introduce

metrici dedicate:

Fidelitatea (faithfulness) măsoară proporția afirmațiilor din răspuns care pot fi deduse din contextul recuperat. O fidelitate de 1.0 indică un răspuns complet susținut de dovezile furnizate.

Relevanța răspunsului (answer relevancy) măsoară în ce grad răspunsul adresează întrebarea originală, penalizând răspunsurile complete factual, dar tangențiale față de întrebare.

Precizia contextului și recuperarea contextului evaluează calitatea recuperării din perspectiva generării: câte dintre fragmentele recuperate sunt cu adevărat necesare și câte dintre fragmentele necesare au fost recuperate [24]. Aceste metrice sunt centrale pentru sistemele orientate spre fidelitate [8] și constituie principalele criterii de comparare în demonstratorul din această teză.

1.5.4 Dificultăți în evaluarea extragerii relațiilor

Evaluarea extragerii relațiilor se realizează cu scorurile micro și macro F_1 calculate pe tipurile de relații. Micro- F_1 ponderează fiecare instanță egal, favorizând tipurile de relații frecvente. Macro- F_1 tratează fiecare tip de relație egal, indiferent de frecvența sa.

Există două dificultăți structurale. Prima este distribuția dezechilibrată a tipurilor de relații în seturile de date existente, care face ca micro- F_1 să fie adesea înșelătoare [25]. A doua este problema falselor negative: relații corecte neanotate în setul de referință duc la subestimarea performanței reale. Re-DocRED [25] a demonstrat că DocRED [26] conținea un număr semnificativ de astfel de false negative.

Metricile de fidelitate nu reflectă direct corectitudinea tipului de relație extras: un sistem poate fi fidel față de contextul recuperat, dar contextul însuși poate conține relații ambigue sau greșit formulate. Prin urmare, evaluarea completă a unui sistem RAG pentru extragerea relațiilor necesită atât metrice de fidelitate, cât și metrice specifice extragerii.

1.6 Baze de date

1.6.1 Seturi de date pentru sisteme RAG

SQuAD [27] (Stanford Question Answering Dataset) leagă întrebări de pasaje din Wikipedia, răspunsul reprezentând un fragment de text contiguu extras din pasaj. Setul conține 100.000 de întrebări construite de adnotatori umani, ceea ce îl face un standard pentru evaluarea recuperării și extracției.

Natural Questions [28] conține întrebări reale adresate motorului de căutare Google, perechi cu pagini Wikipedia. Spre deosebire de SQuAD, întrebările nu sunt construite cu pasaje în față, ceea ce le face mai reprezentative pentru scenariile reale de utilizare. Setul de date include atât răspunsuri scurte, cât și lungi.

TriviaQA [29] este un set de perechi întrebare-răspuns colectate din resurse trivia, însoțite de documente justificative găsite independent prin căutare web. Dificultatea suplimentară față de SQuAD este că documentele justificative nu sunt garantat să conțină răspunsul.

HotpotQA [30] este un benchmark care necesită raționament pe mai multe documente simultan. Fiecare întrebare este adnotată cu faptele justificative din mai multe articole Wikipedia, forțând sistemele să realizeze recuperare și raționament multi-hop. Este relevant pentru extragerea relațiilor care implică mai multe entități intermediare.

CRAG [31] (Comprehensive RAG Benchmark) conține 4.409 perechi QA generate cu API-uri de căutare web și de grafuri de cunoștințe. Acoperă cinci domenii și opt categorii de întrebări

cu niveluri variabile de dinamism temporal, ceea ce îl face util pentru evaluarea robusteții față de informații care se schimbă în timp.

1.6.2 Seturi de date pentru extragerea relațiilor

T-REx [32] este un set de aliniament la scară largă între rezumatele Wikipedia și tripletele Wikidata: 11 milioane de triplete aliniate cu 3,09 milioane de rezumate. Structura sa face T-REx potrivit pentru evaluarea sistemelor care combină recuperarea din text liber cu validarea față de grafuri de cunoștințe. Reprezintă setul de date utilizat în partea experimentală a acestei teze.

WebNLG [33] furnizează perechi date-text pornind de la triplete RDF din DBpedia. Setul de date are aplicații atât în extragerea relațiilor, cât și în verbalizarea datelor structurate, deoarece fiecare triplet este însoțit de fraze naturale care îl exprimă.

SemEval-2010 Task 8 [34] este un benchmark de clasificare a relațiilor semantice între perechi nominale la nivel de propoziție, cu nouă tipuri de relații direcționale plus o clasă negativă. Este utilizat frecvent ca referință pentru compararea metodelor de extragere a relațiilor la nivel de propoziție.

TACRED [35] este un set de date de mari dimensiuni construit din articole de știri și rapoarte ale agenției Associated Press, cu 41 de tipuri de relații și o clasă negativă predominantă. Dezechilibrul accentuat între clase face evaluarea macro- F_1 mai informativă decât micro- F_1 .

DocRED [26] extinde extragerea relațiilor la nivel de document, necesitând raționament inter-propoziții. Un număr semnificativ de relații implică entități menționate în propoziții diferite, ceea ce face DocRED mai dificil decât seturile sentence-level.

Re-DocRED [25] corectează problema falselor negative din DocRED prin reannotare manuală sistematică. Studiul original a arătat că o parte considerabilă din relațiile corecte din DocRED nu fuseseră adnotate, ceea ce subestima performanța modelelor evaluate pe setul original.

Set de date	Sarcină	Nivel	Caracteristici principale
SQuAD	QA extractiv	Propoziție	Fragment de text ca răspuns
Natural Questions	QA	Document	Întrebări reale, răspuns scurt/lung
HotpotQA	QA multi-hop	Multi-document	Fapte justificative adnotate
CRAG	QA factual	Web / GC	4.409 perechi, dinamism temporal
T-REx	Extragere relații	Document	Aliniament text-triplete Wikidata
WebNLG	Extragere/verbalizare	Triplete RDF	Perechi date-text
SemEval-2010 T8	Extragere relații	Propoziție	Relații nominale, 9 tipuri
TACRED	Extragere relații	Propoziție	41 tipuri de relații
DocRED	Extragere relații	Document	Raționament inter-propoziții
Re-DocRED	Extragere relații	Document	Corectare false negative

Tabel 1.1: Comparatie a seturilor de date pentru evaluarea RAG și extragerea relațiilor [25–28, 30–35].

1.7 Limitări și provocări

Calitatea recuperării. Documentele irelevante sau parțial relevante induc în eroare modelul generativ, ducând la halucinații chiar și atunci când există o sursă externă de informație [2, 14]. Recuperarea fixă, neadaptivă, nu poate evalua sau corecta automat rezultatele slabe [8].

Fragmentele care conțin una dintre entitățile căutate, dar nu și relația relevantă, sunt o sursă frecventă de erori în extragerea relațiilor.

Integrarea contextelor lungi. Creșterea numărului de pasaje recuperate nu îmbunătățește neapărat performanța. Pasajele irelevante din contexte lungi degradează calitatea textului generat [19]. Fenomenul, denumit uneori *lost in the middle*, arată că modelele generative tind să ignore informațiile aflate la mijlocul unui context lung, chiar dacă acestea sunt relevante.

Limitări ale datelor și metricilor. Seturile de date pentru extragerea relațiilor au distribuții dezechilibrate și conțin false negative [25]. Metricile de fidelitate nu reflectă direct corectitudinea tipului de relație extras [24]. Construirea de seturi de date de calitate la nivel de document este costisitoare și lentă.

Dificultăți în extragerea relațiilor. Identificarea unei relații necesită adesea agregarea dovezilor din mai multe propoziții sau documente, ceea ce presupune un raționament interpropoziție dificil de realizat într-un singur pas de recuperare [17, 26]. Relațiile implicite, care nu apar formulat explicit în text, reprezintă o provocare suplimentară.

Costul computațional. Indexarea densă, recuperarea hibridă, rerankingul cu cross-encoder și generarea cu LLM-uri sunt toate operații care consumă resurse semnificative [5, 12]. Abordările adaptive și active implică recuperare iterativă, cu mai multe apeluri la model pe interogare [15, 16]. Metodele bazate pe grafuri adaugă costul construirii și actualizării grafului de cunoștințe [13, 18]. În absența unor resurse computaționale ample, aceste cerințe limitează scala la care sistemele pot fi evaluate sau folosite în producție.

1.8 Concluzii

Capitolul de față a prezentat contextul teoretic al sistemelor RAG aplicate extragerii relațiilor între entități. Au fost discutate limitările cunoștințelor parametrice ale modelelor lingvistice mari [2, 3] și motivul pentru care augmentarea cu recuperare externă reprezintă o soluție practică, urmată de o taxonomie specifică acestei teze, construită pe criteriile tipului de mecanism de recuperare, al modului de integrare recuperare-generare și al sursei de cunoștințe.

Analiza literaturii arată că robustețea sistemelor RAG depinde în primul rând de calitatea recuperării și de capacitatea de a evalua adaptiv dovezile recuperate [8, 14]. Extragerea relațiilor beneficiază cel mai mult de structurile de grafuri de cunoștințe și de abordările cu raționament la nivel de document [17, 18, 26]. Provocările deschise rămân integrarea contextelor lungi [19], limitările metricilor și datelor [24, 25] și costul computațional [12].

Lucrarea se plasează la intersecția optimizării recuperării cu dezvoltarea de mecanisme generative care integrează explicit dovezile recuperate. Demonstratorul propus compară Naive RAG cu Advanced RAG, pornind de la observația că optimizările pre- și post-recuperare conduc la îmbunătățiri măsurabile ale fidelității și relevanței răspunsurilor [2, 24], aspecte cu impact direct asupra corectitudinii relațiilor extrase.

Capitolul 2

Rezultate experimentale

Demonstratorul implementat în cadrul acestei lucrări pune față în față două arhitecturi RAG pe același set de date și același model lingvistic, cu scopul de a izola contribuția etapelor de recuperare avansată față de varianta de bază. Implementarea rulează local, pe hardware propriu, fără dependență de servicii externe sau cloud. În continuare sunt descrise mai întâi deciziile de proiectare și procesul de dezvoltare, urmate de evaluarea cantitativă a celor două sisteme și interpretarea calitativă a rezultatelor.

Arhitectura sistemului și deciziile de proiectare

Ambele pipeline-uri partajează aceeași bază: datele sunt indexate offline, iar la momentul interogării se execută recuperarea și generarea. Diferența stă exclusiv în complexitatea etapei de recuperare.

Modelul lingvistic folosit este Qwen3-8B [2], rulat local prin Ollama. Alegerea unui model open-source rulat local, în loc de un API extern, a fost motivată de controlul total asupra promptului de sistem și de eliminarea oricărei latențe de rețea. Qwen3 suportă un mod de gândire (“thinking mode”) care poate fi dezactivat prin directiva `/no_think` în prompt, ceea ce a fost util pentru expansiunea interogărilor, unde nu este nevoie de raționament extins.

Modelul de embedding ales este `all-MiniLM-L6-v2` [5], un model compact cu 22M de parametri, antrenat pentru sarcini de regăsire semantică. Raportul calitate-viteză al acestui model îl face o alegere standard în sistemele RAG care nu au constrângeri hardware severe. Baza de date vectorială este ChromaDB, configurată cu spațiu cosinus pentru calculul similarității.

Setul de date

Setul de date folosit este SQuAD (Stanford Question Answering Dataset) [27], versiunea 1.1, split-ul de antrenare. SQuAD conține perechi întrebare-răspuns extrase din articole Wikipedia, unde răspunsul este un fragment de text din pasajul asociat. Această proprietate face posibilă o evaluare precisă a recuperării: se poate verifica dacă fragmentul recuperat conține răspunsul corect și provine din articolul corect, fără a depinde de judecata unui model suplimentar. La ingestie, paragrafele duplicate au fost eliminate, rezultând un corpus de paragrafe unice stocate ca fragmente individuale în ChromaDB.

Procesul de dezvoltare

Implementarea a pornit cu varianta minimă funcțională: indexarea datelor SQuAD în ChromaDB și un pipeline Naive conectat la o interfață Streamlit. Interfața permite interogarea simultană a ambelor sisteme și afișarea comparativă a răspunsurilor, a fragmentelor recuperate și a scorurilor asociate.

Componentele Advanced RAG au fost adăugate treptat. Prima extensie a fost recuperarea sparse cu BM25 [6]. Inițial s-a folosit biblioteca `rank_bm25`, dar aceasta a fost înlocuită

cu **bm25s**, mult mai rapidă prin tokenizare batch și indexare optimizată. Indexul BM25 se calculează o singură dată și se salvează pe disc.

Configurarea cross-encoder-ului a necesitat câteva iterații: au fost testate mai multe modele, ajungând la **cross-encoder/ms-marco-MiniLM-L-6-v2**, care oferă un compromis bun între precizia re-scorării și latența de inferență. Pool-ul de candidați pentru reranking a crescut de la 20 la 40 pe parcursul dezvoltării, după ce s-a observat că un pool mai mic tăia uneori fragmentele relevante înainte ca cross-encoder-ul să le poată evalua.

Expansiunea interogărilor folosește Qwen3-1.7B, ales deliberat pentru dimensiunea redusă. Reformularea nu necesită raționament complex, dar viteza contează: apelul LLM se află pe calea critică a fiecărei interogări, iar un model de 1.7B parametri este de câteva ori mai rapid decât 8B.

Scriptul de benchmark a fost construit cu suport pentru mai multe seed-uri și checkpoint după fiecare seed, astfel încât o întrerupere să nu piardă rezultatele deja calculate. Testele de semnificație statistică (Wilcoxon și McNemar) au fost adăugate ulterior, pentru a valida că diferențele observate nu sunt artefacte ale eșantionării aleatoare.

2.1 Analiză cantitativă

2.1.1 Configurarea experimentului

Evaluarea a rulat pe câte 150 de întrebări selectate aleator din split-ul de antrenare SQuAD [27], pe 4 seed-uri distincte (42, 1042, 2042, 3042), totalizând 600 de evaluări per sistem. Utilizarea mai multor seed-uri permite estimarea variabilității rezultatelor și raportarea mediei cu deviația standard.

Metricile calculate sunt:

- **Context Recall@5**: fracția de întrebări pentru care cel puțin unul din cei $K=5$ candidați recuperați conține răspunsul corect și provine din articolul corect. Condiția privind sursa este necesară pentru a preveni potrivirile false pe fragmente scurte și frecvente (de exemplu, “mai” sau “SUA”) care pot apărea în contexte complet diferite [27].
- **Mean Reciprocal Rank (MRR)**: inversul rangului primului fragment relevant, mediat pe toate întrebările [5]. MRR penalizează sistemele care plasează fragmentul corect pe poziții inferioare în lista de rezultate.
- **Answer F1**: scorul F_1 la nivel de token între răspunsul generat și răspunsurile de referință SQuAD, calculat ca maxim peste toate variantele de referință disponibile.
- **Exact Match**: proporția de răspunsuri generate care conțin textul de referință ca subșir exact, după normalizarea majusculilor și a semnelor de punctuație.
- **Latența medie de recuperare**: timpul de procesare a interogării în milisecunde. Pentru Advanced RAG, aceasta include apelul LLM pentru expansiunea interogării, recuperarea hibridă paralelă și re-scorarea cu cross-encoder.

Pentru testarea semnificației statistice s-au aplicat două teste pereche pe cele 600 de înregistrări combinate din toate seed-urile. Testul Wilcoxon signed-rank a fost aplicat pe metricile continue (MRR și Answer F1), iar testul McNemar pe metricile binare (Context Recall și Exact Match) [2].

2.1.2 Rezultate

Rezultatele agregate pe cele 4 seed-uri sunt prezentate în Tabelul 2.1. Advanced RAG depășește Naive RAG pe toate metricile de calitate, cu o creștere de 14,4 puncte procentuale la Context Recall (de la 83,5% la 95,5%), 25,7 puncte la MRR (de la 0,719 la 0,904) și 10,4 puncte la Exact Match (de la 64,2% la 74,5%). Answer F1 crește mai modest, cu 7,3% relativ (de la 0,191 la 0,205).

Tabel 2.1: Rezultate comparative Naive RAG vs. Advanced RAG pe SQuAD ($N = 600$ per sistem, medie $\pm$ deviație standard pe 4 seed-uri)

Metrică	Naive RAG	Advanced RAG
Context Recall@5	0,835 $\pm$ 0,006	0,955 $\pm$ 0,015
MRR	0,719 $\pm$ 0,013	0,904 $\pm$ 0,023
Exact Match	0,642 $\pm$ 0,027	0,745 $\pm$ 0,045
Answer F1	0,191 $\pm$ 0,008	0,205 $\pm$ 0,015
Latență medie (ms)	15,6 $\pm$ 0,1	769,3 $\pm$ 20,8

Figurile 2.1 și 2.2 ilustrează grafic diferențele pentru metricile de recuperare și de calitate a răspunsurilor. Figura 2.3 prezintă comparativ latența medie de procesare, iar Figura 2.4 oferă o vedere de ansamblu a tuturor metricilor de calitate.

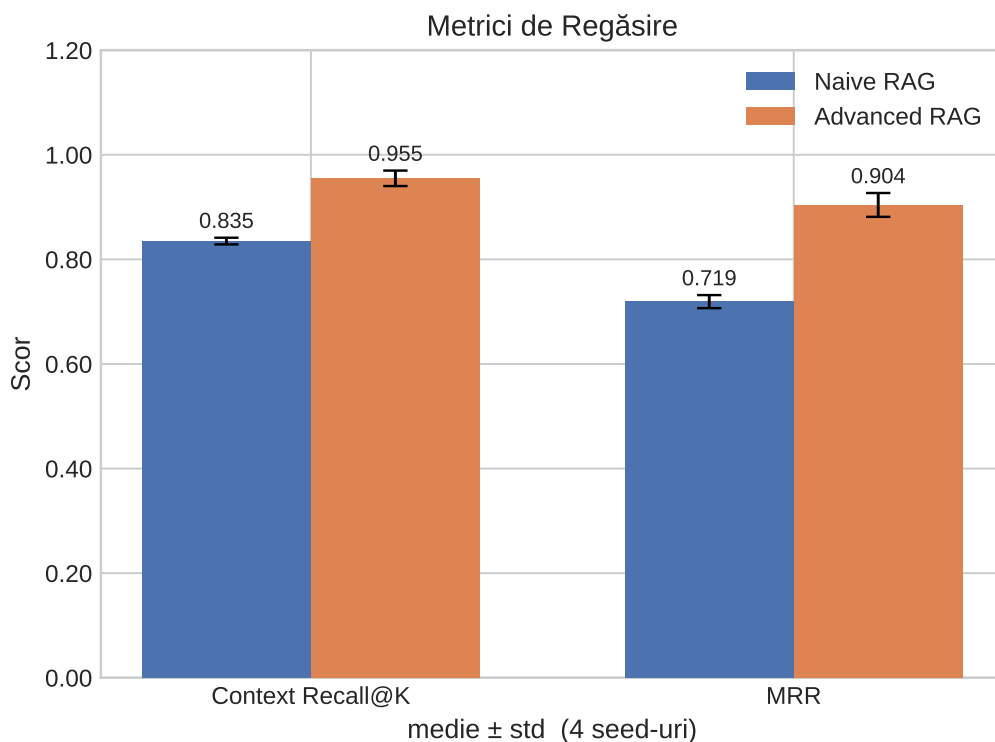


Figura 2.1: Metrici de recuperare: Context Recall@5 și MRR pentru Naive RAG și Advanced RAG. Barele de eroare reprezintă deviația standard calculată pe cele 4 seed-uri.

2.1.3 Semnificație statistică

Toate diferențele observate sunt semnificative statistic la $p < 0,05$, cu marje considerabile față de prag (Tabelul 2.2).

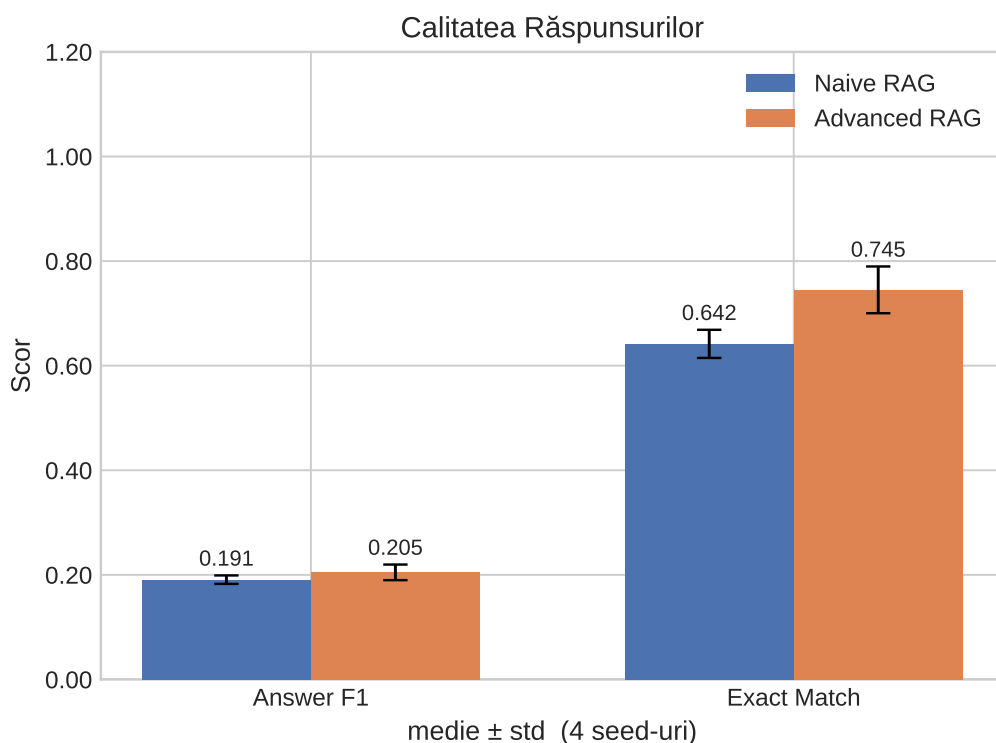


Figura 2.2: Calitatea răspunsurilor: Answer F1 și Exact Match pentru Naive RAG și Advanced RAG.

Tabel 2.2: Teste de semnificație statistică (Advanced RAG vs. Naive RAG, $N = 600$ perechi)

Metrică	Test	Valoarea p	Observații
MRR	Wilcoxon	$6,5 \times 10^{-27}$	—
Answer F1	Wilcoxon	$1,8 \times 10^{-3}$	—
Context Recall	McNemar	$7,7 \times 10^{-20}$	Adv. mai bun: 74; Naive mai bun: 2
Exact Match	McNemar	$4,3 \times 10^{-10}$	Adv. mai bun: 82; Naive mai bun: 20

Valoarea p extrem de mică pentru MRR ($6,5 \times 10^{-27}$) indică o diferență sistematică, nu un artefact al eșantionării. Testul McNemar pentru Context Recall arată că există 74 de întrebări unde Advanced RAG recuperează fragmentul corect, iar Naive RAG nu reușește, față de doar 2 în situația inversă. Raportul de 37:1 în favoarea sistemului avansat confirmă că îmbunătățirile în recuperare nu provin din cazuri marginale, ci reprezintă un avantaj consistent.

2.2 Analiză calitativă

2.2.1 Compromisul dintre calitate și latență

Cel mai vizibil cost al arhitecturii avansate este latența de recuperare: 769 ms față de 16 ms pentru varianta de bază, o creștere de aproximativ 50 de ori. Această diferență are câteva surse distincte. Expansiunea interogărilor implică un apel LLM (Qwen3-1.7B), care durează câteva sute de milisecunde chiar și pentru un model de 1.7B parametri. Recuperarea densă și BM25 rulează în paralel pe toate cele trei formulări (originală plus două alternative), ceea ce limitează parțial impactul, dar cross-encoder-ul introduce o latență suplimentară pentru re-scorarea celor 40 de candidați.

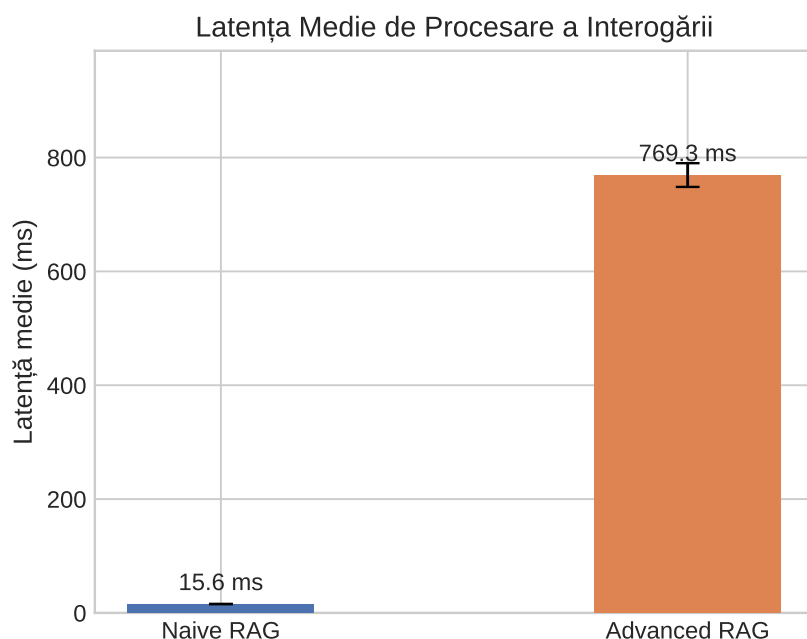


Figura 2.3: Latența medie de procesare a interogării în milisecunde. Latența Advanced RAG include apelul LLM pentru expansiunea interogării, recuperarea hibridă paralelă și re-scorarea cu cross-encoder.

În contextul aplicațiilor RAG reale, o latență de recuperare de 770 ms rămâne acceptabilă, deoarece generarea unui răspuns cu un model de 8B parametri durează oricum mai multe secunde. Totuși, în scenarii unde se doresc timpi de răspuns sub o secundă, overhead-ul expansiunii interogărilor devine o constrângere reală și ar putea fi eliminat printr-o versiune fără expansiune, care combină recuperarea hibridă fără reformulare [2].

2.2.2 Unde ajută recuperarea hibridă

Diferența cea mai pronunțată dintre cele două sisteme apare la metricile de recuperare, nu la cele de generare. Creșterea cu 14,4 puncte la Context Recall înseamnă că, în 12 din 100 de cazuri unde Naive RAG eșuează, Advanced RAG găsește totuși fragmentul corect. MRR crescut de la 0,719 la 0,904 arată că, pe lângă faptul că găsește mai des fragmentul corect, îl și plasează pe poziții mai înalte.

Recuperarea hibridă (densă + BM25) are un avantaj clar pe interogările cu termeni specifici: date, numere, denumiri proprii, acronime. Modelul de embedding codifică înțeles semantic, dar poate dilua termeni rari într-un spațiu vectorial continuu unde similaritatea cosinus favorizează documente tematic apropiate, nu neapărat lexical identice. BM25 [6] nu are această problemă: un termen care apare exact o dată în corpus va fi regăsit cu precizie dacă apare și în interogare. Expansiunea interogărilor contribuie la recall prin acoperirea variantelor de formulare, reducând dependența de formularea exactă a întrebării [2].

2.2.3 Decalajul dintre recuperare și generare

Îmbunătățirile la recuperare nu se traduc proporțional în îmbunătățiri la generare. Context Recall crește cu 14 puncte procentuale, dar Answer F1 crește cu 7% relativ, iar Exact Match cu 10 puncte procentuale.

Explicația principală este că SQuAD a fost conceput ca benchmark pentru citire extractivă, unde modelul ar trebui să extragă textual un fragment din pasaj. Modelele generative moderne

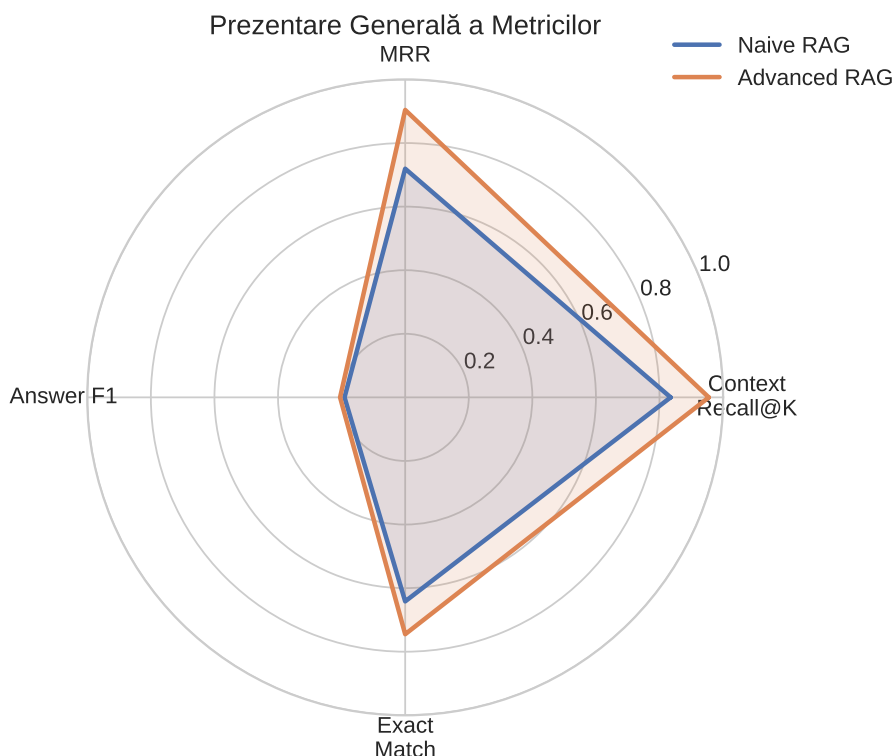


Figura 2.4: Vedere de ansamblu comparativă a metricilor de calitate (Context Recall@5, MRR, Answer F1, Exact Match). Latența nu este inclusă deoarece se află pe o scară diferită (milise-cunde față de 0–1).

tind să parafrazeze răspunsul mai degrabă decât să îl reproducă cuvânt cu cuvânt, ceea ce scade Answer F1 față de valoarea maximă posibilă, chiar dacă răspunsul este corect semantic. O a doua explicație este că Naive RAG recuperează corect fragmentul în 83,5% din cazuri, ceea ce înseamnă că modelul generativ primea oricum contextul corect pentru cea mai mare parte a întrebărilor. Câștigul din Advanced RAG vine tocmai din cazurile dificile, adică interogările cu termeni rari sau cu răspunsuri scurte greu de localizat prin similaritate semantică pură, unde generarea oricum eșuează mai des.

2.2.4 Constrângerea de groundare și refuzurile

Ambele pipeline-uri folosesc un prompt de sistem care constrânge modelul să răspundă exclusiv din contextul furnizat și să refuze explicit dacă informația lipsește. Exact promptul de refuz este: *“The provided context does not contain enough information to answer this question.”* Această decizie de proiectare este direct motivată de principiul fidelității contextuale descris în [8]: un sistem care preferă să refuze decât să inventeze este mai fiabil decât unul care generează răspunsuri plauzibile dar incorecte.

Refuzurile nu pot fi evaluate cu Answer F1 sau Exact Match, deoarece nu conțin conținut factual comparabil cu referința. Advanced RAG generează mai puține refuzuri decât Naive RAG, pentru că recuperează mai frecvent fragmentul corect și îl plasează pe primele poziții, astfel că modelul primește mai rar un context din care nu poate răspunde.

2.2.5 Observații calitative din interfața Streamlit

Interfața Streamlit permite observarea directă a diferențelor de comportament fără a parcurge metricile agregate.

Pe întrebări cu termeni numerici sau coduri specifice (“Câte specii...”, “În ce an...”, “Care este codul...”), Naive RAG returnează adesea fragmente tematic relevante care nu conțin numărul exact. Advanced RAG găsește fragmentul corect prin BM25, care tratează numerele ca termeni distincți și nu le diluează în spațiu vectorial.

Când răspunsul nu există în corpus, ambele sisteme refuză explicit. Diferența este că Advanced RAG produce mai rar răspunsuri parțiale construite din context adiacent irelevant, deoarece cross-encoder-ul elimină candidații slab relevanți înainte de generare.

Pe întrebări tematice generale (“Despre ce este...”, “Care este rolul...”), ambele sisteme produc rezultate similare. Recuperarea densă funcționează bine pe similitudine semantică, iar avantajul hibridizării este mai puțin pronunțat în aceste cazuri.

Aceste observații sunt consistente cu rezultatele cantitative: câștigurile Advanced RAG sunt mai mari pe cazurile dificile (termeni rari, răspunsuri numerice) și mai mici pe interogările tematice generale, unde recuperarea densă funcționează deja bine [2, 6].

Concluzii

Rezultate obținute

Lucrarea a urmărit două obiective paralele: un studiu al literaturii de specialitate privind sistemele de tip Retrieval-Augmented Generation (RAG) și implementarea unui demonstrator care compară o arhitectură de bază cu una avansată pe un set de date public.

Din studiul de literatură (Capitolul 1) au reieșit câteva concluzii clare. Problema fundamentală pe care o adresează RAG rămâne aceeași ca la apariția termenului în 2020 [3]: modelele lingvistice mari au cunoștințe fixate la momentul antrenării și generează răspunsuri incorecte atunci când nu dețin informația solicitată. RAG rezolvă această limitare nu prin reantrenare, ci prin recuperarea documentelor relevante la momentul interogării și condiționarea generării pe aceste documente. Diversitatea abordărilor din literatură, de la Naive RAG la arhitecturi modulare și grafuri de cunoștințe [2, 4], reflectă că nu există o soluție universală: alegerea arhitecturii depinde de tipul sursei de date, de latența acceptabilă și de natura sarcinii.

Studiul a identificat că metodele de recuperare hibridă (densă + BM25 [6]) și reranking-ul cu cross-encoder aduc cele mai consistente îmbunătățiri față de recuperarea densă pură, în special pe interogări cu termeni rari sau numerici. Metodele adaptive precum Self-RAG [8] și CRAG [14] adaugă un nivel de auto-evaluare a calității recuperării, reducând halucinațiile cauzate de documente irelevante. Pentru extragerea relațiilor între entități, abordările bazate pe grafuri de cunoștințe [17] sunt cele mai potrivite structural, deoarece relațiile sunt reprezentate explicit și pot fi traversate direct.

Din evaluarea experimentală (Capitolul 2) rezultatele arată că Advanced RAG depășește Naive RAG pe toate metricile de calitate, cu diferențe semnificative statistic ($p \ll 0,05$ în toate testele). Context Recall@5 a crescut de la 83,5% la 95,5% (+14,4 puncte procentuale), MRR de la 0,719 la 0,904 (+25,7 puncte), iar Exact Match de la 64,2% la 74,5% (+10,4 puncte). Testul McNemar pentru Context Recall a arătat că există 74 de întrebări unde Advanced RAG recuperează fragmentul corect iar Naive RAG nu, față de doar 2 în situația inversă, un raport de 37:1 în favoarea arhitecturii avansate.

Costul acestor îmbunătățiri este latența: Advanced RAG necesită 770 ms per interogare față de 16 ms pentru Naive RAG, o diferență de aproximativ 50 de ori. Cea mai mare parte a acestui overhead provine din apelul LLM pentru expansiunea interogărilor. În aplicații unde latența de recuperare poate fi ascunsă parțial de latența de generare (care durează oricum câteva secunde pentru modele de 8B parametri), compromisul este acceptabil. Îmbunătățirile la Answer F1 au fost mai modeste (7,3% relativ), ceea ce este de așteptat: SQuAD este un benchmark extractiv, iar modelele generative modifică formularea răspunsului chiar și atunci când contextul corect este disponibil.

Contribuții originale

Contribuția principală a lucrării este demonstratorul implementat, disponibil ca proiect open-source, care compară Naive RAG și Advanced RAG pe același set de date și același model lingvistic, izolând contribuția etapelor de recuperare. Sistemul rulează local prin Ollama și ChromaDB, fără dependență de servicii externe.

Din punct de vedere tehnic, Advanced RAG adaugă patru optimizări față de varianta de bază. Recuperarea hibridă combină recuperarea densă (all-MiniLM-L6-v2 [5] + ChromaDB) cu recuperarea sparse (BM25 [6] prin `bm25s`), listele rezultate fuzionând prin Reciprocal Rank Fusion [7]. Expansiunea interogărilor folosește Qwen3-1.7B pentru a genera două reformulări ale interogării originale, reducând dependența de formularea exactă. Reranking-ul cu cross-encoder (`ms-marco-MiniLM-L-6-v2`) re-scorează cele mai bune 40 de candidate și returnează primele 5. Ambele sisteme refuză explicit când contextul recuperat nu conține informația solicitată, în loc să genereze răspunsuri plauzibile dar incorecte [8].

Scriptul de benchmark suportă evaluare pe mai multe seed-uri cu checkpoint după fiecare run, teste de semnificație statistică (Wilcoxon și McNemar) și generare automată de figuri. Interfața Streamlit permite testarea interactivă a ambelor sisteme și vizualizarea comparativă a răspunsurilor.

Taxonomia din Capitolul 1 a fost adaptată pentru extragerea relațiilor între entități, nu pentru domeniul RAG în ansamblu, ceea ce limitează acoperirea dar face clasificarea mai directă pentru înțelegerea alegerii arhitecturale din demonstrator.

Perspective de dezvoltare ulterioară

O limitare concretă a evaluării actuale este că s-a realizat pe SQuAD, un set de date de tip QA extractiv. Lucrarea urmărește aplicarea RAG la extragerea relațiilor între entități, pentru care T-REx [32] (aliniament Wikipedia-Wikidata) sau DocRED [26] sunt mai potrivite structural, deoarece conțin triplete (subiect, predicat, obiect) explicit adnotate. Adaptarea benchmark-ului pentru aceste seturi rămâne pasul imediat următor.

Evaluarea RAGAS [24], cu metrici de fidelitate (faithfulness) și relevanță a răspunsului (answer relevancy), nu a putut fi finalizată în limitele de timp disponibile din cauza costului computațional ridicat (modelul judecător rulează câte un apel LLM per pereche întrebare-răspuns). O evaluare completă ar adăuga perspectiva calității fundamentării în context, independentă de potrivirea textuală cu referința.

Pe latura arhitecturală, GraphRAG [17] și HippoRAG [18] indexează explicit relațiile dintre entități și le utilizează la recuperare, ceea ce le face candidați direcți pentru extragerea relațiilor față de pipeline-urile actuale bazate pe fragmente de text. O comparație extinsă cu aceste metode ar clarifica în ce măsură structura relațională explicită aduce beneficii față de recuperarea hibridă pe text nestructurat.

Latența Advanced RAG (770 ms per interogare) provine în cea mai mare parte din apelul LLM pentru expansiunea interogărilor. O variantă fără expansiune, care păstrează recuperarea hibridă și cross-encoder-ul, ar reduce latența la câteva zeci de milisecunde cu o pierdere relativ mică la recall. Această variantă ar fi utilă în scenarii cu constrângeri stricte de timp de răspuns.

În fine, evaluarea actuală folosește Qwen3-8B ca model generativ, fără fine-tuning. Modele mai mari sau antrenate specific pe extragerea relațiilor pot produce îmbunătățiri la Answer F1 independent de arhitectura de recuperare. Separarea contribuției modelului generativ de contribuția pipeline-ului de recuperare rămâne o direcție deschisă pentru cercetare ulterioară.

Bibliografie

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [2] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-Augmented Generation for Large Language Models: A survey, 2024.
- [3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.
- [4] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. Graph Retrieval-Augmented Generation: A survey. *arXiv preprint arXiv:2408.08921*, 2024.
- [5] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020.
- [6] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [7] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.
- [8] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations*, 2023.
- [9] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1762–1777, 2023.
- [10] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. REALM: Retrieval-Augmented language model pre-training. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 3929–3938, 2020.
- [11] Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, pages 874–880, 2021.

- [12] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, et al. Improving language models by retrieving from trillions of tokens. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 2206–2240, 2022.
- [13] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *International Conference on Learning Representations (ICLR)*, 2024.
- [14] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation. *arXiv preprint arXiv:2401.15884*, 2024.
- [15] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 7036–7050, 2024.
- [16] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7969–7992, 2023.
- [17] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- [18] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hippo-RAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [19] Bowen Jin, Jinsung Yoon, Jiawei Han, and Serkan O Arik. Long-context LLMs meet RAG: Overcoming challenges for long inputs in RAG. *arXiv preprint arXiv:2410.05983*, 2024.
- [20] Lang Mei, Siyu Mo, Zhihan Yang, and Chong Chen. A survey of multimodal retrieval-augmented generation. *arXiv preprint arXiv:2504.08748*, 2025.
- [21] Peng Xia, Kangyu Zhu, Haoran Li, Tianze Wang, Weijia Shi, Sheng Wang, Linjun Zhang, James Zou, and Huaxiu Yao. MMed-RAG: Versatile multimodal RAG system for medical vision language models. *arXiv preprint arXiv:2410.13085*, 2024.
- [22] Haoran Hao, Jiaming Han, Changsheng Li, Yu-Feng Li, and Xiangyu Yue. RAP: Retrieval-augmented personalization for multimodal large language models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 14538–14548, 2025.
- [23] Jun Chen, Dannong Xu, Junjie Fei, Chun-Mei Feng, and Mohamed Elhoseiny. Document haystacks: Vision-language reasoning over piles of 1000+ documents. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 24817–24826, 2025.
- [24] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated evaluation of Retrieval Augmented Generation. *arXiv preprint arXiv:2309.15217*, 2023.
- [25] Qingyu Tan, Lu Xu, Lidong Bing, Hwee Tou Ng, and Sharifah Mahani Aljunied. Revisiting DocRED – addressing the false negative problem in relation extraction. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 8472–8487, 2022.

- [26] Yuan Yao, Deming Ye, Peng Li, Xu Han, Yankai Lin, Zhenghao Liu, Zhiyuan Liu, Lixin Huang, Jie Zhou, and Maosong Sun. DocRED: A large-scale document-level relation extraction dataset. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 764–777, 2019.
- [27] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In Jian Su, Kevin Duh, and Xavier Carreras, editors, *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, Austin, Texas, November 2016. Association for Computational Linguistics.
- [28] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, et al. Natural Questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics (TACL)*, 7:453–466, 2019.
- [29] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1601–1611, 2017.
- [30] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2369–2380, 2018.
- [31] Xiao Yang, Kai Sun, Hao Xin, Yushi Sun, Nikita Bhalla, Xiangsen Chen, Sajal Choudhary, Rongze Daniel Gui, Ziran Will Jiang, Ziyu Jiang, et al. CRAG – comprehensive RAG benchmark. In *Advances in Neural Information Processing Systems (NeurIPS) – Datasets and Benchmarks Track*, 2024.
- [32] Hady Elsahar, Pavlos Vougiouklis, Arslan Remaci, Christophe Gravier, Jonathon Hare, Frederique Laforest, and Elena Simperl. T-REx: A large scale alignment of natural language with knowledge base triples. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC)*, pages 3448–3452, 2018.
- [33] Claire Gardent, Anastasia Shimorina, Shashi Narayan, and Laura Perez-Beltrachini. Creating training corpora for NLG micro-planners. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 179–188, 2017.
- [34] Iris Hendrickx, Su Nam Kim, Zornitsa Kozareva, Preslav Nakov, Diarmuid Ó Séaghdha, Sebastian Padó, Marco Pennacchiotti, Lorenza Romano, and Stan Szpakowicz. SemEval-2010 task 8: Multi-way classification of semantic relations between pairs of nominals. In *Proceedings of the 5th International Workshop on Semantic Evaluation (SemEval)*, pages 33–38, 2010.
- [35] Yuhao Zhang, Victor Zhong, Danqi Chen, Gabor Angeli, and Christopher D. Manning. Position-aware attention and supervised data improve slot filling. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 35–45, 2017.

Anexa 1

Anexa 2